COSC 81/281: Problem Set 4: Perception and Learning

Due Monday, October 26 (11:59pm ET)

Problem sets are graded **pass/fail on completion**, and there are no late days. The problem sets are where the learning happens, and they are the best available practice for the midterm and final exam. Submit to **Gradescope** (https://www.gradescope.com/courses/1372528) and assign each question to a page in your submission. Questions are welcome on **Slack**. Starred (★) questions are required for COSC 281 students and optional (ungraded) for COSC 81 students.

PS4 accepts no late submissions. Solutions release the same evening the assignment is due (Monday, October 26 (11:59pm ET)), so that you have them in hand while you prepare for the exam (Thursday, October 29 (in class)). Submit what you have by the deadline.

> **AI Policy.** You may use any AI tools you like to assist you with your assignments, and you should, as that is likely how you will work for the rest of your career. That being said, the written portions are practice for the in-class midterm and exam, which are closed book and on paper. You should strive to answer these yourself and use AI as a tutor and not an oracle.

Please include an AI Tool Disclosure Below

What to submit. Submit to **Gradescope** (https://www.gradescope.com/courses/1372528):

1. **Written work (PDF).** Include your Part 1 answers, the Sobel figure from Task 1, and the image comparison and explanation from Task 2. P1(d) and P4(b) are graduate-only. You may write on this handout or use `ps4/handout.tex`, entering answers in the `studentanswer` blocks after each `SOLUTION HERE` marker. Assign each question to its page or pages when uploading.
2. **Code.** Submit only `conv.py`. Run `python ps4/code/autograder.py` from the course repository root. The color experiment uses provided code and has no autograder submission.

Part 1: Written problems

Show your calculations or explain your reasoning in each response area. The written questions can be answered without running the programming tasks.

P1: Convolution and image boundaries

The matrix K is a filter, and I is a grayscale image whose entries are pixel intensities.

$$K = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}, \qquad I = \begin{bmatrix} 4 & 4 & 4 \\ 4 & 12 & 4 \\ 4 & 4 & 4 \end{bmatrix}.$$

Apply the filter by multiplying each entry of K by the image entry in the same position of the current 3×3 patch, then summing the nine products. Use stride 1, meaning that consecutive filter positions differ by one pixel. Fill each response grid with the requested pixel values, and show your calculations below it. Leave noninteger results as fractions.

(a) With no padding, the filter fits in exactly one location. Compute the convolution output by multiplying corresponding entries and adding the nine products.

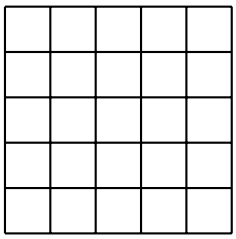
Answer / work

Output



(b) Padding means adding entries around the image boundary before applying the filter. Add one row or column of zeros on each of the four sides of I. Fill every cell of the 5×5 grid with the resulting padded image, including the zeros. Do not apply the filter yet.

Answer / work



Padded image

P1 (continued)

(c) Using the padded image from part (b), compute the output at the top-left corner. Show the nonzero products in that weighted sum. State the center output as well. Does it change from part (a)? You do not need to compute the other seven output entries.

Answer / work

	Top-left output	Center output
Value		

(d) ★ (required for 281; optional for 81) Suppose the border is instead filled by repeating the nearest original pixel value. Explain which output entries can change and why. Why is the choice of padding an assumption about the image, rather than only a way to control the output size? No additional calculation is required.

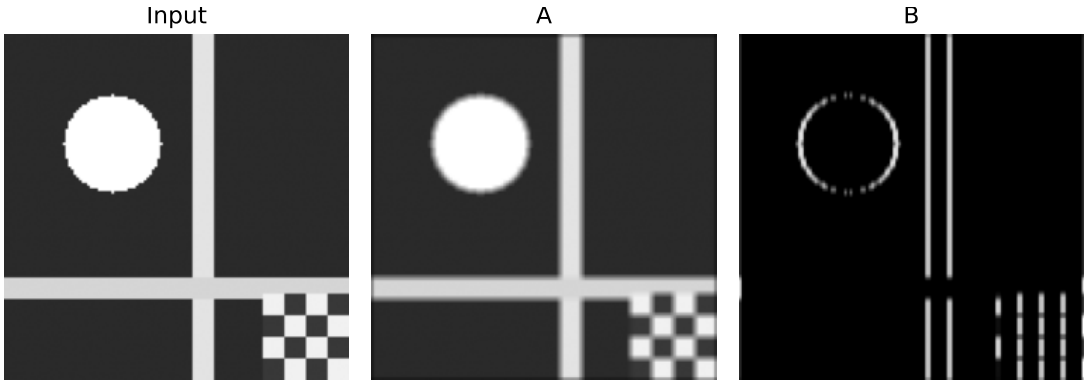
Answer / work

P2: What does a filter detect?

The figure shows one grayscale image and two filtered outputs, A and B. The two filters are

$$K_1 = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad K_2 = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

Each output is displayed using the magnitude of its values, scaled so that its largest magnitude is white. Focus on which image features remain visible rather than comparing brightness between panels.



(a) Match each filter to output A or B and explain both choices. For K_1, compare the left and right coefficients on a constant patch and on a patch whose right side is brighter. For K_2, use the fact that its positive coefficients sum to one. Explain what happens at an edge.

Answer / work

P3: Following an image through a network

A grayscale image stores one intensity at each pixel. A *channel* is one grid of values over the image. Thus an 8×8 grayscale image has height 8, width 8, and one channel. An RGB image has three such grids, one each for red, green, and blue. The input in this question is grayscale.

A convolutional layer applies several learned filters to the same input. Each filter produces its own grid of responses, called an output channel. These learned channels need not represent colors. One might respond to an edge, while another responds to a different pattern. Here the layer uses two 3×3 filters, stride 1, and one pixel of zero padding on every side. Each filter combines one input patch into one output value.

Max pooling is a separate layer applied after the convolution. It divides each channel into nonoverlapping 2×2 blocks and replaces each block by its largest value. For example, $\begin{bmatrix} 1 & 5 \\ 3 & 2 \end{bmatrix}$ becomes the single value 5. It processes each channel independently and has no learned parameters.

(a) Start with the $8 \times 8 \times 1$ input. After padding, each spatial dimension is 10. A width-3 filter fits at $10 - 3 + 1$ horizontal positions, and the same count applies vertically. Fill the table with the dimensions after convolution and after pooling. The number of convolution channels equals the number of filters. Pooling halves each spatial dimension but keeps the channels separate.

Answer / work

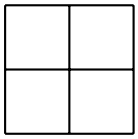
Layer	Height	Width	Channels
Convolution			
Max pooling			

P3 (continued)

(b) Practice max pooling on the following 4×4 portion of one channel. Use nonoverlapping 2×2 blocks and write the four maxima in the output grid. State one kind of information lost by this operation.

$$\begin{bmatrix} 1 & 5 & 2 & 0 \\ 3 & 2 & 4 & 1 \\ 0 & 7 & 3 & 8 \\ 6 & 1 & 2 & 5 \end{bmatrix}$$

Answer / work



(c) A *fully connected layer* uses every value from the preceding layer as an input to each of its outputs. First, place the pooled $4 \times 4 \times 2$ values in a list $x_1, \ldots, x_N$. This rearrangement is called flattening and adds no learned parameters. The layer has three outputs, each computed as

$$y_j = w_{j1}x_1 + \cdots + w_{jN}x_N + b_j, \qquad j = 1, 2, 3.$$

Here each weight w_{ji} and each bias b_j is a learned number. Find N, then count the weights and biases for one output and for all three outputs. Count only this fully connected layer.

Answer / work

P4: Evaluating an imitation policy

A team trains a steering policy for the Dubins car using behavior cloning. The training data contain observations and steering commands from successful expert laps of a slalom course. The learned policy passes the first two gates, then drifts wide on the third. Its steering becomes progressively less effective until it leaves the course. The demonstrations contain no examples of recovering from this deviation.

(a) Identify a learning-related cause of this failure. In two or three sentences, explain why the errors can grow during a rollout even when the policy predicts steering accurately on held-out expert data.

Answer / work

(b) ★ (required for 281; optional for 81) Suppose collecting additional expert demonstrations is not possible. Propose one modification that could reduce failures during deployment. Explain why it could help and identify one limitation. Answer in two or three sentences. Multiple answers may be acceptable when their assumptions are stated and justified.

Answer / work

Part 2: Image filtering experiments

Run all commands from the course repository root with the course Python environment active. The only file to implement and submit is `ps4/code/conv.py`. Task 2 uses a complete, provided program.

Task 1: Convolution and Sobel

A grayscale image is a two-dimensional array with shape (H, W). Each entry is an intensity. A filter is a smaller array that assigns weights to a local image patch. At each position, multiply entries in the same positions of the patch and filter, then sum the products to obtain one output value.

The file `conv.py` contains two functions to complete. The image loader, grayscale conversion, filter coefficients, and plotting are provided. You do not need to edit any other file.

1. In `convolve2d(image, kernel, pad=0, stride=1)`, convert the input arrays to floating-point values with `np.asarray`. The image has shape (H, W) and the filter has odd square shape (K, K).
2. Add `pad` rows or columns of zeros to each side of the image using `np.pad`. The padded image has shape $(H + 2\,\texttt{pad}, W + 2\,\texttt{pad})$.
3. Allocate a floating-point output array with dimensions

   $$H_{\text{out}} = (H + 2\,\texttt{pad} - K)\ //\ \texttt{stride} + 1, \quad W_{\text{out}} = (W + 2\,\texttt{pad} - K)\ //\ \texttt{stride} + 1.$$

   Here `//` is integer division. Stride gives the number of input pixels between neighboring filter positions.
4. Use nested loops over output rows i and columns j. The corresponding patch starts at row $r = i\,\texttt{stride}$ and column $c = j\,\texttt{stride}$ in the padded image. Select it using `padded[r:r+K, c:c+K]`.
5. Multiply the patch and `kernel` entry by entry. Use `np.sum` to store their weighted sum in output entry (i, j). Return the completed array. Do not use library filtering functions from SciPy, OpenCV, or a neural-network framework.

For a small check, an image $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ with a 3×3 filter whose entries are all $1/9$, no padding, and stride 1 should produce the single value 5. A 4×4 image with a 3×3 filter and padding 1 should produce a 4×4 output at stride 1 and a 2×2 output at stride 2.

Task 1 (continued)

Sobel responses. The supplied `SOBEL_X` filter compares intensities to the left and right of each pixel. `SOBEL_Y` compares intensities above and below it. They produce signed responses g_x and g_y, which indicate local intensity changes in the two directions. Large response magnitudes often occur along visible edges. Complete `sobel(image)` as follows.

1. Call your `convolve2d` with `SOBEL_X`, padding 1, and stride 1 to obtain g_x.
2. Call it again with `SOBEL_Y` and the same padding and stride to obtain g_y.
3. Compute $\sqrt{g_x^2 + g_y^2}$ separately at every pixel using NumPy array operations. Return `gx, gy, magnitude`. All three arrays must have the same shape as the input image.

Run

```
python ps4/code/conv.py
python ps4/autograder.py
```

The first command loads the supplied simulator image, converts it to grayscale, calls your functions, and saves `ps4/code/plots/edges.png`. The second command runs the public checks. Submit only `conv.py` to the autograder.

PDF response. Include `edges.png` and identify one boundary that produces a strong response. No additional code or written derivation is required.

Answer / work

Task 2: Color and illumination

This experiment uses `ps4/code/hsv_detector.py`, which is fully implemented. You do not need to write a detector or submit code for this task.

Grayscale, RGB, and HSV. A grayscale image stores one intensity per pixel. An RGB image stores three values per pixel, for red, green, and blue, so its array has shape $(H, W, 3)$. Those three channels together describe a color. Converting RGB to grayscale combines them into one intensity and discards some color information.

HSV is another representation of the same RGB color, not a new sensor. Hue describes the color family and is represented here by an angle in $[0, 360)$. Saturation describes how far the color is from gray, and value describes its brightness. Both lie in $[0, 1]$ in the provided code. HSV separates these quantities for thresholding, but it does not make colors invariant to illumination or camera settings. Hue is especially unreliable when saturation is very low.

Provided detector. The program converts each image to HSV and selects pixels whose hue, saturation, and value meet fixed thresholds. It then keeps the largest connected region of selected pixels. The colored overlay shows that region, and a marker shows its center. This is a color rule, not a shape recognizer. A similarly colored background can be selected instead of the object.

1. Run `python ps4/code/hsv_detector.py`. The program uses the supplied simulator images and saves overlays in `ps4/code/plots/`. Open them to see which pixels the rule selects.

2. Choose a red, orange, or yellow object and photograph the same object in two lighting conditions, such as daylight and warm indoor light, or ordinary light and strong shadow. Keep the detector thresholds fixed. The exercise concerns the change in appearance, not whether the object is literally a cone.

3. Transfer the photos to the computer running your code. Create a folder named `photos` inside `ps4/code/`, then copy the files there using your file manager. If your code runs in a hosted notebook, use its file browser's upload button and place the files in that same folder. Use JPEG or PNG images. Export HEIC phone photos as JPEG or PNG first.

4. Rename the files `daylight.jpg` and `indoor.jpg`, or substitute their actual names in this command:

   ```
   python ps4/code/hsv_detector.py
   ps4/code/photos/daylight.jpg ps4/code/photos/indoor.jpg
   ```

   Enter the two displayed lines as one command. Put quotation marks around any path containing spaces. The program prints each output path.

If taking or transferring photos is not possible, run `python ps4/code/hsv_detector.py -lighting-demo`. It uses a supplied simulator frame to create controlled dim and color-shifted examples, then applies the same detector. These are synthetic changes, not measurements of a real camera. Use the original and one changed image, and state that you used this option.

Task 2 responses

(a) Include the two overlays you compared and label their lighting conditions. State whether they are your photographs or the provided synthetic lighting examples.

Answer / work

(b) In a short paragraph, explain how the appearance and selected region changed. Relate one observation to brightness, saturation, color, shadows, or white balance. What does the comparison show about the limits of using fixed color thresholds? Successful detection in both images is also a valid result.

Answer / work